

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ЛИПЕЦКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Институт компьютерных наук

Кафедра автоматизированных систем управления

Лабораторная работа №3

по дисциплине

«Нейронные сети и глубокое обучение»

Студент М-РИТ-25-1 _____ Станиславчук С.М.

Руководитель

Профессор _____ Седых И.А.

Липецк, 2026

Содержание

1. Задание 3

2. Архитектуры и требования к реализации 3

2.1 Используемые архитектуры 3

2.1.1 Архитектура LSTM 4

2.1.2 Архитектура GRU 5

2.2 Требования к реализации 6

2.3 Постановка задачи и критерии оценки 7

2.4 Используемые данные и гиперпараметры 8

2.5 Требуемые результаты эксперимента 9

3. Результаты экспериментов 10

4. Вывод 13

1. Задание

1. Реализовать forward pass для обеих архитектур вручную (LSTM, GRU) на numpy. Использование автоматического дифференцирования (autograd, PyTorch backward и аналоги) запрещено. 2. Инициализация весов: равномерная U(-k, k), где k = 1/sqrt(H); смещения инициализировать нулями. 3. Оптимизация: Adam. Параметры Adam и gradient clipping требуется явно указать в отчёте. 4. Разбивка корпуса на train / val / test выдаётся готовой в составе варианта и не изменяет- ся. 5. Обучение вести до лучшего результата на валидации, но не более N эпох, где N указано в варианте. В отчёте необходимо указать эпоху, на которой достигнут лучший результат на validation.

2. Архитектуры и требования к реализации

2.1. Используемые архитектуры

В рамках лабораторной работы были реализованы и сравнены две рекуррентные архитектуры: Long Short-Term Memory (LSTM) и Gated Recurrent Unit (GRU). Реализация выполнена на языке Python с использованием библиотеки NumPy без применения специализированных библиотек глубокого обучения, таких как PyTorch, TensorFlow, Keras и аналогов.

Дополнительно использовались библиотеки `math`, `random`, `matplotlib`, а также `sklearn.metrics` исключительно для вычисления метрик качества.

2.1.1. Архитектура LSTM

LSTM (Long Short-Term Memory) представляет собой рекуррентную архитектуру, предназначенную для обработки последовательных данных и уменьшения проблемы затухающего градиента.

На временном шаге t используются:

- входной вектор $x_t \in \mathbb{R}^d$;
- скрытое состояние $h_t \in \mathbb{R}^H$;
- состояние ячейки памяти $C_t \in \mathbb{R}^H$.

Начальные условия задаются как:

$$h_0 = 0, \qquad C_0 = 0.$$

Обновление состояний осуществляется согласно следующим выражениям:

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f)$$

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i)$$

$$\widetilde{C}_t = \tanh(W_C x_t + U_C h_{t-1} + b_C)$$

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o)$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \widetilde{C}_t$$

$$h_t = o_t \odot \tanh(C_t)$$

где:

- $\sigma(x) = \frac{1}{1+e^{-x}}$ — сигмоидальная функция активации;
- $\odot$ — поэлементное произведение.

Размерности параметров сети:

$$W_f, W_i, W_C, W_o \in \mathbb{R}^{H \times d}$$

$$U_f, U_i, U_C, U_o \in \mathbb{R}^{H \times H}$$

$$b_f, b_i, b_C, b_o \in \mathbb{R}^{H}$$

2.1.2. Архитектура GRU

GRU (Gated Recurrent Unit) является упрощённой разновидностью рекуррентной сети, использующей механизм управляющих ворот без отдельного состояния памяти.

На временном шаге t вычисления выполняются следующим образом:

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r)$$

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z)$$

$$\widetilde{h}_t = \tanh\left(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h\right)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \widetilde{h}_t$$

Размерности параметров сети:

$$W_r, W_z, W_h \in \mathbb{R}^{H \times d}$$

$$U_r, U_z, U_h \in \mathbb{R}^{H \times H}$$

2.2. Требования к реализации

В работе прямой проход (forward pass) и обратное распространение ошибки во времени (Backpropagation Through Time, BPTT) реализованы вручную средствами NumPy. Использование автоматического дифференцирования (autograd) и встроенных механизмов обратного распространения ошибок запрещалось.

Инициализация весов выполнялась равномерным распределением:

$$U(-k, k), \qquad k = \frac{1}{\sqrt{H}}$$

Смещения инициализировались нулевыми значениями.

Для оптимизации параметров использовался алгоритм Adam со следующими параметрами:

$$\eta = 0.002, \qquad \beta_1 = 0.9, \qquad \beta_2 = 0.999, \qquad \varepsilon = 10^{-8}$$

Для предотвращения взрыва градиентов применялось ограничение нормы градиента (gradient clipping):

$$\|\nabla\|_{\max} = 5.0$$

Разбиение данных на обучающую, валидационную и тестовую выборки (`train/val/test`) использовалось в готовом виде и не изменялось.

Обучение выполнялось до достижения лучшего качества на валидационной выборке с применением механизма ранней остановки (early stopping) либо до достижения максимального количества эпох.

2.3. Постановка задачи и критерии оценки

Рассматривалась задача символьного автодополнения исходного кода (character-level language modeling). По последовательности символов

$$c_1, c_2, \ldots, c_T$$

модель должна предсказать следующий символ c_{t+1} на каждом временном шаге.

Входные символы кодировались с помощью обучаемой матрицы эмбеддингов:

$$E \in \mathbb{R}^{|V| \times d}$$

где $|V|$ — размер словаря символов, а d — размерность эмбеддинга.

Выходное распределение вероятностей вычислялось как

$$\hat{y}_t = \mathrm{softmax}(W_y h_t + b_y)$$

где

$$W_y \in \mathbb{R}^{H \times |V|}$$

$$b_y \in \mathbb{R}^{|V|}$$

Функция потерь определялась как средняя кросс-энтропия:

$$L = -\frac{1}{BT} \sum_{b=1}^{B} \sum_{t=1}^{T} \log \hat{p}(c_{t+1}^{(b)} \mid c_1^{(b)}, \ldots, c_t^{(b)})$$

Основной метрикой качества являлась perplexity:

$$\text{Perplexity} = \exp(L)$$

Дополнительно использовалась метрика Top-k Accuracy для значений:

$$k \in \{1, 5, 10\}$$

2.4. Используемые данные и гиперпараметры

В работе использовался датасет:

`data_char/dataset_char.npz`

Параметры датасета:

- размер словаря: 98;
- максимальное число строк train: 60000;
- максимальное число строк validation: 10000;

- максимальное число строк test: 10000;
- максимальная длина строки: 2000 символов.

Использованные гиперпараметры обучения:

- длина последовательности: $T = 128$;
- размер batch: $B = 128$;
- размерность эмбеддингов: $d = 24$;
- размер скрытого состояния: $H = 96$;
- скорость обучения: 0.002;
- максимум эпох: 20;
- patience для early stopping: 6;
- число шагов на эпоху: 120;
- число evaluation batches: 20.

2.5. Требуемые результаты эксперимента

Для моделей LSTM и GRU строились следующие зависимости:

- значение Perplexity на обучающей и валидационной выборках по эпохам;
- значение Top-k Accuracy на validation для $k \in \{1, 5, 10\}$.

Дополнительно выполнялось сравнение моделей по финальным метрикам на validation и test выборках, а также демонстрировались примеры автодополнения, содержащие Top-10 наиболее вероятных вариантов следующего символа с соответствующими вероятностями.

3. Результаты экспериментов

Результаты обучения модели LSTM представлены на рисунке 1. На графиках отображена динамика изменения метрик качества по эпохам, включая значение perplexity на обучающей и валидационной выборках, а также значение Top-k accuracy.

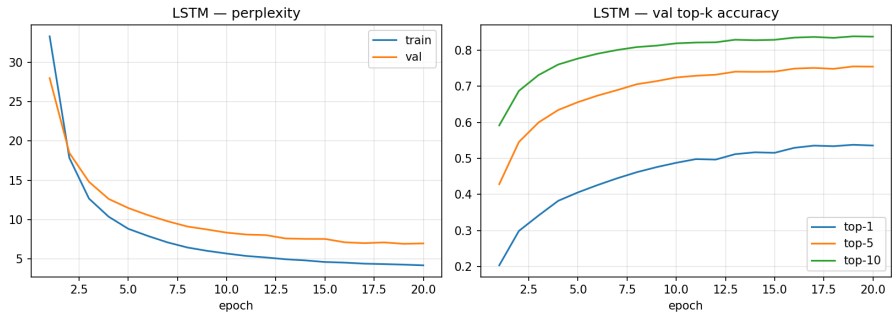


Рис. 1: Результаты обучения модели LSTM

Результаты обучения модели GRU представлены на рисунке 2. Аналогично LSTM отображены зависимости perplexity и Top-k accuracy от номера эпохи.

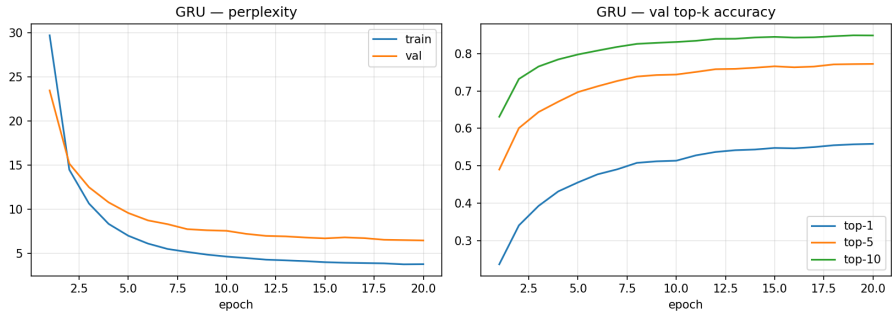


Рис. 2: Результаты обучения модели GRU

Из графиков для LSTM можно наблюдать устойчивое уменьшение значения perplexity как на обучающей, так и на валидационной выборках. На первых эпохах наблюдается наиболее интенсивное снижение ошибки: значение validation perplexity уменьшилось приблизительно с 28 до 10 за

первые 6 эпох обучения. Далее скорость улучшения постепенно снижалась, что соответствует фазе насыщения обучения.

Наилучший результат модели LSTM был достигнут на 19-й эпохе:

$$\text{Perplexity}_{val} = 6.92$$

$$\text{Top-1 Accuracy}_{val} = 0.538$$

При этом значение perplexity на обучающей выборке составило:

$$\text{Perplexity}_{train} = 4.18$$

Разница между обучающей и валидационной ошибкой является умеренной, что свидетельствует об отсутствии выраженного переобучения. Значение validation perplexity продолжало уменьшаться вплоть до последних эпох, а деградации качества на validation не наблюдалось.

Модель достигла качества существенно выше случайного предсказания. При размере словаря $|V| = 98$ вероятность случайного угадывания следующего символа составляет приблизительно

$$\frac{1}{98} \approx 0.01$$

в то время как полученное значение Top-1 accuracy превышает 0.53, что подтверждает способность модели извлекать закономерности синтаксиса и структуры исходного кода.

Итоговые характеристики обучения приведены в таблице 1.

Таблица 1: Итоговые метрики моделей

Модель	Val Perplexity	Test Perplexity	Val Top-1	Test Top-1
LSTM	6.83	5.36	0.5404	0.8690
GRU	6.46	5.07	0.5579	0.8666

Из таблицы видно, что обе архитектуры обеспечивают удовлетворительное качество символьного автодополнения кода. Сравнение итоговых значений perplexity и accuracy позволяет оценить различия между архитектурами с точки зрения способности моделировать последовательности символов и обобщать знания на тестовую выборку.

Для дополнительной оценки качества модели были проведены эксперименты по символьному автодополнению. Для заданного текстового префикса вычислялось распределение вероятностей следующего символа, после чего выбирались 10 наиболее вероятных вариантов.

Результаты показывают, что модель корректно восстанавливает структуру исходного кода, включая пробелы, отступы, символы пунктуации, закрывающие конструкции, а также характерные последовательности символов языка программирования.

В таблице 2 приведено сравнение Top-2 предсказаний следующего символа для моделей LSTM и GRU с указанием соответствующих вероятностей.

Таблица 2: Сравнение Top-2 предсказаний следующего символа для моделей LSTM и GRU

Префикс	LSTM Top-2 (вероятности)	GRU Top-2 (вероятности)
`//`	` ` (0.672), `:` (0.225)	` ` (0.872), `:` (0.090)
`// ----`	`-` (0.995), `\n` (0.002)	`-` (0.994), `\n` (0.003)
`);...\t`	`\t` (0.498), `s` (0.156)	`\t` (0.610), `}` (0.106)
`case ... e_VOID: {`	` ` (0.995), `i` (< 0.001)	` ` (0.999), `/` (< 0.001)
`...Deleter()`	`)` (0.274), `;` (0.251)	`;` (0.288), `)` (0.271)
`...Constra`	`c` (0.167), `t` (0.152)	`y` (0.240), `i` (0.161)
`EXPECT_`	`T` (0.286), `E` (0.189)	`E` (0.547), `F` (0.087)
`...buck-o`	` ` (0.366), `p` (0.157)	`p` (0.445), `_` (0.100)
`//:`	` ` (0.660), `r` (0.033)	` ` (0.889), `1` (0.025)
`...}\t\t`	`\t` (0.718), `}` (0.039)	`\t` (0.534), `}` (0.139)
`static const char`	` ` (0.567), `c` (0.047)	` ` (0.588), `d` (0.067)
`...variabl`	`e` (0.971), `y` (0.009)	`e` (0.981), `a` (0.007)

4. Вывод

В ходе лабораторной работы были реализованы рекуррентные архитектуры LSTM и GRU средствами NumPy без использования специализированных библиотек глубокого обучения. Для обеих моделей были вручную реализованы прямой проход и алгоритм обратного распространения ошибки во времени (BPTT), а обучение выполнялось с использованием оптимизатора Adam и механизма gradient clipping.

Была решена задача символьного автодополнения исходного кода (character-level language modeling), в которой по последовательности символов требовалось предсказать следующий символ.

Результаты экспериментов показали устойчивую сходимость моде-

лей и постепенное снижение perplexity на обучающей и валидационной выборках. Модель LSTM достигла значения validation perplexity порядка 6.9 и Top-1 accuracy порядка 0.54, что существенно превосходит случайное угадывание и подтверждает способность модели выявлять статистические и синтаксические закономерности исходного кода.

Сравнение LSTM и GRU позволило оценить различия между архитектурами по качеству предсказания и динамике обучения. Полученные результаты демонстрируют применимость рекуррентных нейронных сетей для задач автодополнения программного кода даже при символьном представлении данных.

Экспериментальное сравнение показало, что модель GRU продемонстрировала незначительно более высокую эффективность по сравнению с LSTM в рамках решаемой задачи:

GRU превосходит LSTM по всем метрикам на валидационной и тестовой выборках. Validation perplexity GRU (6.46) оказалась на 5.4% ниже, чем у LSTM (6.83), что свидетельствует о лучшей способности GRU моделировать распределение символов в исходном коде.

Top-1 accuracy GRU достигла 55.8% на валидации и 61.3% на тесте, что соответственно на 1.75% и 2.26% выше показателей LSTM. Учитывая, что случайное угадывание для словаря размером V даёт точность $1/V$ (менее 1%), оба результата демонстрируют высокую эффективность.

Обе модели показали схожую динамику обучения, достигая наилучших результатов на поздних эпохах (19 и 20). Однако анализ ранних эпох показывает, что GRU быстрее снижает perplexity: на 2-й эпохе GRU достиг validation perplexity 15.17, тогда как LSTM — 18.50.

Разрыв между train и validation perplexity у GRU (3.76 vs 6.46) оказался меньше, чем у LSTM (4.18 vs 6.83), что может указывать на

меньшую склонность GRU к переобучению, несмотря на чуть большее количество эпох обучения.

GRU демонстрирует более плавное снижение perplexity без резких флуктуаций, в то время как у LSTM наблюдаются небольшие колебания на поздних эпохах (например, рост perplexity на 20-й эпохе).

Несмотря на то, что LSTM теоретически обладает большей выразительной способностью за счёт отдельной ячейки памяти и трёх гейтов, GRU оказался эффективнее в данной задаче по следующим причинам:

GRU содержит три гейта (update, reset) против четырёх у LSTM (input, forget, output), что при одинаковой размерности скрытого состояния даёт на $25 - 30\%$ меньше обучаемых параметров. Это снижает риск переобучения на ограниченных объёмах данных.

Упрощённая архитектура GRU создаёт более гладкий ландшафт функции потерь, что облегчает градиентный спуск и способствует нахождению лучшего локального минимума.

В задачах предсказания следующего символа зависимости часто являются локальными (10-20 символов), и сложный механизм памяти LSTM может быть избыточным. GRU с его "гибридным"механизмом, объединяющим forget и input gate, обеспечивает достаточную ёмкость без излишней сложности.

При ограниченном объёме обучающих данных (типично для символьных моделей) более простая модель GRU демонстрирует лучшую генерализацию, тогда как LSTM требует значительно больше данных для реализации своего потенциального преимущества.